



SECURITY REVIEW • PREPARED FOR

Smoke Ring

Cross-chain expansion of the SMOKE
token from Base to Solana over LayerZero

DATE

15.09.2026

COMMIT

b6212059

REMEDIATION

8a04b785

Phase & process

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Telegram [@sbsecurity_audits](https://t.me/sbsecurity_audits).

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1 IMPACT

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2 LIKELIHOOD

- **High** - almost certain to happen, easy to perform, or highly incentivized.
- **Medium** - only conditionally possible, but still relatively likely.
- **Low** - requires specific state or little-to-no incentive.

02 EXECUTIVE SUMMARY

Key findings

TOTAL FINDINGS

3

RESOLVED

3 of 3 · 100%

HIGH SEVERITY FINDINGS

0 none found

SEVERITY DISTRIBUTION

☒ Fixed & verified ☒ Acknowledged

CRITICAL 0

HIGH 0

MEDIUM 0

LOW 3

INFORMATIONAL 0

GAS OPT. 0

CLIENT

Smoke Ring

REPOSITORY

Private

METHODS

Manual Review,
AI Review

TIMELINE · SEPTEMBER 2026

Audit · 4 days

Remediation · 2 days

KICK-OFF

09.09.2026

END OF AUDIT

12.09.2026

REMEDIATIONS START

14.09.2026

REMEDIATIONS END

15.09.2026

STARTING COMMIT HASH

b62120593eb0c6f2f21fc0b54bf6c8a53a5ceedd

FINAL COMMIT HASH

8a04b78531cfee61f98fcae3c15f0a176d1f7a4e

CONTRACTS IN SCOPE

contracts/SmokeMinter.sol	programs/smoke_minter/src/lib.rs
programs/smoke_minter/src/errors.rs	programs/smoke_minter/src/events.rs
programs/smoke_minter/src/msg_codec.rs	programs/smoke_minter/src/instructions/mod.rs
programs/smoke_minter/src/instructions/init_store.rs	programs/smoke_minter/src/instructions/set_peer.rs
programs/smoke_minter/src/instructions/set_config.rs	programs/smoke_minter/src/instructions/renounce_admin.rs
programs/smoke_minter/src/instructions/lz_receive.rs	programs/smoke_minter/src/instructions/lz_receive_types_info.rs
programs/smoke_minter/src/instructions/lz_receive_types_v2.rs	programs/smoke_minter/src/instructions/burn.rs
programs/smoke_minter/src/state/mod.rs	programs/smoke_minter/src/state/store.rs
programs/smoke_minter/src/state/peer.rs	programs/smoke_minter/src/state/stats.rs
programs/smoke_nfts_staging/src/lib.rs	programs/smoke_nfts_staging/src/errors.rs
programs/smoke_nfts_staging/src/events.rs	programs/smoke_nfts_staging/src/instructions/mod.rs
programs/smoke_nfts_staging/src/instructions/init.rs	programs/smoke_nfts_staging/src/instructions/set_nft_authority.rs
programs/smoke_nfts_staging/src/instructions/transfer_admin.rs	programs/smoke_nfts_staging/src/instructions/accept_admin.rs
programs/smoke_nfts_staging/src/instructions/handle_mint_funding.rs	programs/smoke_nfts_staging/src/state/mod.rs
programs/smoke_nfts_staging/src/state/store.rs	

03 PROJECT SUMMARY

What Smoke Ring is

SMOKE is a token on Base that is being extended to Solana as a LayerZero OFT. Next to the standard OFT bridge, the project adds a one-way conversion path: a user hands OG_SMOKE to the SmokeMinter contract on Base, 90% of what arrives is burned and the rest goes to the buy and burn address, and a LayerZero message tells the smoke_minter program on Solana to mint SMOKE to the user's wallet, one to one in whole tokens.

The Solana side also ships a small staging vault that holds SMOKE for a future NFT program, and a permissionless burn that keeps a total burned counter in line with the Base token.

SECURITY POSTURE

BEFORE REMEDIATION

B6212059

AFTER REMEDIATION

8A04B785

Unchecked deploy-time configuration

L-01 · L-02

Two values the receiver depends on were taken from the deployer as-is. The SMOKE mint was bound without checking that it has 6 decimals, while the create task defaults to 9, so every bridged amount could land a thousand times too small. The endpoint program used for the replay guard was stored straight from the init parameters, so a wrong value would have turned the clear call into a no-op.

Launch runbook

L-03

The planned hand-off of the mint authority to the final 1-of-2 multisig and the matching smoke_minter config update ran as separate transactions. In between, every Base to Solana delivery fails until the store is repaired, and the stock setauthority task aborted halfway because the freeze authority is already revoked.

All three findings are fixed on the audit-fixes branch and the changes match the recommendations. set_config and the staging init now accept only a classic SPL Token mint with 6 decimals, and both programs dropped the Token-2022 interface types, so a mis-scaled or extension-bearing mint is rejected at binding time. init_store no longer takes an endpoint parameter and always stores the canonical EndpointV2 id, with the test suite loading its endpoint mock at that same id.

A new finalize-authority task creates the final multisig, changes the mint authority, and repoints smoke_minter in one transaction. It checks every account it touches before sending, re-reads the state at finalized commitment afterwards, and only then rewrites the deployment record. LiteSVM and task-level tests cover the new checks and the rotation.

04 RECOMMENDATIONS

Beyond this audit

Measures that extend the protection of this review after launch, independent of any individual finding.

R-01

Revoke the upgrade authority

Non-upgradeable is a deploy-time property on Solana. Revoke the BPF upgrade authority of both programs right after deployment and record the transaction in the runbook.

R-02

Finish the authority hand-off in order

After finalize-authority runs, confirm that no wallet key is still a member of the mint-authority multisig. Only renounce the smoke_minter admin once the mint, the multisig, and the Base peer are confirmed on chain, and renounce the Endpoint delegate at the same time since renounce_admin does not touch it.

R-03

Monitor the conversion path

Watch for failed lz_receive deliveries and payloads left pending on the Solana side, and reconcile total minted on Solana against the MintedToSolana events on Base. Alert on any divergence.

R-04

Key hygiene

The smoke_minter admin, the staging admin, and the Base SmokeMinter owner control every trusted binding in the system. Keep them on hardware signers behind a multisig until they are renounced.

Core contracts

CONTRACT	ROLE
<code>SmokeMinter.sol</code>	Send-only LayerZero OApp on Base. Pulls OG_SMOKE from the caller, burns 90% of what arrives, forwards the rest to the buy and burn address, and sends the 41-byte mint message to Solana. Holds no funds between transactions.
<code>smoke_minter</code>	LayerZero OApp receiver on Solana. Checks the trusted peer, clears the message at the Endpoint, creates the recipient's token account if needed, and mints SMOKE through the mint-authority multisig. Also exposes a permissionless burn with a total burned counter.
<code>smoke_nfts_staging</code>	SMOKE vault for a future NFT program. The admin sets the NFT authority once; that authority can then move SMOKE out of the vault to any receiver. Two-step admin transfer, no LayerZero code.

Privileged roles

smoke_minter admin.

Set at init_store by the build-time initial admin. Sets the trusted Base peer and the SMOKE mint with its multisig, and can renounce itself, which freezes both forever. Also the OApp delegate at the Endpoint. A single key with no transfer path.

SMOKE mint authority.

A 1-of-N SPL Token multisig. At launch its members are the OFT store PDA, the smoke_minter store PDA, and the project wallet; the final state drops the wallet and keeps the two program PDAs. Any member can mint on its own.

Staging admin.

Sets the NFT authority exactly once and can hand the admin seat to another key through a two-step transfer.

NFT authority.

A PDA of the future NFT program. Its signature alone moves any amount of SMOKE out of the staging vault.

Base SmokeMinter owner.

OApp owner and LayerZero delegate on Base. Sets the Solana peer, the enforced options, and the delegate. The contract has no pause, no upgrade path, and holds no funds.

07 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
L-01	SMOKE mint decimals are never checked and the create task defaults to 9	LOW	● FIXED
L-02	<code>endpoint_program</code> is taken from <code>init_store</code> params without validation	LOW	● FIXED
L-03	Non-atomic SMOKE authority rotation delays Base-to-Solana credits	LOW	● FIXED

● LOW SEVERITY · 3 FINDINGS

L-01

LOW

● FIXED

SMOKE mint decimals are never checked and the create task defaults to 9

The wire format carries `amount` in 6 decimal units and `lz_receive` mints that value as is.

https://github.com/SB-Security/audit-2026-09-SMOKE-SOL/blob/master/programs/smoke_minter/src/msg_codec.rs#L21-L24

```
/// Decodes a remote-mint payload into `(recipient, amount)`.
///
/// `amount` is in 6-decimal SMOKE base units and is minted as-is (local decimals
/// on Solana are 6, so there is no sd↔ld conversion).
```

`set_config` checks the mint key and its authority, but never `decimals`. `lz_receive` pins only the address and the token program.

https://github.com/SB-Security/audit-2026-09-SMOKE-SOL/blob/master/programs/smoke_minter/src/instructions/set_config.rs#L38-L47

```
require!(
    ctx.accounts.smoke_mint.mint_authority
    =
    anchor_spl::token_2022::spl_token_2022::solana_program::program_option::COption::Some(
        params.mint_authority_multisig
    ),
    SmokeMinterError::InvalidMint
);

ctx.accounts.store.smoke_mint = params.smoke_mint;
ctx.accounts.store.mint_authority_multisig = params.mint_authority_multisig;
```

The mint is created by `lz:oft:solana:create`, whose local decimals default to 9. The OFT program does its own decimal conversion, so a 9 decimal mint works for OFT transfers and nothing else in the system flags it.

<https://github.com/SB-Security/audit-2026-09-SMOKE-SOL/blob/master/tasks/solana/createOFT.ts#L137-L138>

```
.addOptionalParam('localDecimals', 'Token local decimals (default=9)',
DEFAULT_LOCAL_DECIMALS, devtoolsTypes.int)
.addOptionalParam('sharedDecimals', 'OFT shared decimals (default=6)',
DEFAULT_SHARED_DECIMALS, devtoolsTypes.int)
```

IMPACT

If the mint is created with the default 9 decimals, every bridged amount is credited 1000 times too small. A user who converts 100 OG_SMOKE on Base receives 0.1 SMOKE. `burn` and `total_burned` are off by the same factor. The program cannot detect or undo this. It takes a deploy time mistake by the trusted operator.

RECOMMENDATION

Assert the decimals when the mint is bound. The program cannot work with any other value, so the check cannot break a correct deployment.

L-01 • CONTINUED

https://github.com/SB-Security/audit-2026-09-SMOKE-SOL/blob/master/programs/smoke_minter/src/instructions/set_config.rs#L38

```
require!(ctx.accounts.smoke_mint.decimals == 6, SmokeMinterError::InvalidMint);
```

Add the same check to the staging `init` . Document `--local-decimals 6` in the `lz:oft:solana:create` runbook.

Optionally also require the mint to be owned by the classic SPL Token program, which is the only program `create0FT` accepts for a new mint. That rules out Token-2022 extensions such as transfer fees, transfer hooks, permanent delegates, or accounts that start frozen.

L-02

LOW

● FIXED

endpoint_program is taken from init_store params without validation

init_store stores whatever endpoint_program the caller passes. Only None falls back to the canonical id.

https://github.com/SB-Security/audit-2026-09-SMOKE-SOL/blob/master/programs/smoke_minter/src/instructions/init_store.rs#L44-L46

```
let store = &mut ctx.accounts.store;
store.admin = Some(params.admin);
store.endpoint_program = params.endpoint_program.unwrap_or(ENDPOINT_ID);
```

The value is the target of the clear CPI in lz_receive. That call is the replay guard and the only proof that the message was verified. Nothing writes the field again.

https://github.com/SB-Security/audit-2026-09-SMOKE-SOL/blob/master/programs/smoke_minter/src/instructions/lz_receive.rs#L65-L67

```
let accounts_for_clear = &ctx.remaining_accounts[0..Clear::MIN_ACCOUNTS_LEN];
let _ = oapp::endpoint_cpi::clear(
    ctx.accounts.store.endpoint_program,
```

The wiring task always passes null. The override exists only so the tests can point at endpoint-test-mock.

IMPACT

A wrong or malicious value at init turns clear into a no-op, so anyone could deliver arbitrary mint messages. The value is permanent. Only INITIAL_ADMIN can call init_store.

RECOMMENDATION

Drop the parameter and always use ENDPOINT_ID. The test build already ships a second copy of the mock at the canonical id, so the None path stays testable.

https://github.com/SB-Security/audit-2026-09-SMOKE-SOL/blob/master/programs/smoke_minter/src/instructions/init_store.rs#L46

```
store.endpoint_program = ENDPOINT_ID;
```

If the override must stay for tests, keep it behind a cargo feature that is off in the shipped build.

L-03

LOW

● FIXED

Non-atomic SMOKE authority rotation delays Base-to-Solana credits

The planned launch flow replaces the initial 1-of-3 mint-authority multisig (OFT Store PDA, `smoke_minter` Store PDA, and the project wallet) with a final 1-of-2 containing only the two program PDAs.

These two values must change together:

```
SMOKE mint authority           → final 1-of-2
smoke_minter Store.mint_authority_multisig → final 1-of-2
```

The current tasks update them in separate transactions. `createMintAuthorityMultisig` first creates the replacement multisig:

`multisig.ts`, lines 24-76

`lz:oft:solana:setauthority` then changes the mint authority, while `smoke:minter:set-config` updates the `smoke_minter` Store separately:

`setAuthority.ts`, lines 135-204

`tasks/smoke/index.ts`, lines 182-201

After the mint-authority transaction confirms, the live state is:

```
SMOKE mint authority:           final 1-of-2
smoke_minter Store expected authority: initial 1-of-3
```

`lz_receive` requires the supplied authority to match both values:

`lz_receive.rs`, lines 31-44

```
address = store.mint_authority_multisig @ SmokeMinterError::InvalidMint,
constraint = smoke_mint.mint_authority
    = COption::Some(mint_authority_multisig.key())
    @ SmokeMinterError::InvalidMint
```

The old multisig fails the live-authority check, while the new multisig fails the Store-address check. Every `smoke_minter` delivery attempted before `set_config` confirms fails with `InvalidMint`.

The team confirmed that freeze authority will already be `None`. This makes the current task fail after changing the mint authority: it submits `MintTokens` first, then reaches the following check while handling `FreezeAccount`:

```
if (!mintAuthRet.freezeAuthority) {
    throw new Error(`Mint ${mintStr} has no freeze authority`)
}
```

The task therefore reports failure after the mint-authority change has already finalized. It also leaves the Store and `OFT.json` pointing to the old multisig.

This requires the project operator to run the planned authority rotation. An external user cannot create the mismatch.

IMPACT

A Base-to-Solana conversion executed during this window fails before the Endpoint `clear` CPI. The user receives no Solana SMOKE at that time, but the payload remains pending and can be retried after the Store is repaired.

`lz_receive.rs`, lines 57-79

Repair requires the active `smoke_minter` Store admin to call:

L-03 · CONTINUED

```
smoke:minter:set-config --multisig <final-1-of-2>
```

The explicit `--multisig` is needed because `setauthority` updates neither the Store nor `OFT.json`; running `set-config` without it still reads the old address.

The normal OFT route continues working because the OFT Store PDA belongs to both multisigs. This issue delays the custom Base OG_SMOKE conversion route; it does not consume the pending payload or create an unauthorized mint.

PROOF OF CONCEPT

The repository already contains a LiteSVM test for the same inconsistent state:

```
smoke_minter.test.ts, lines 543-553
```

The test changes the live mint authority without updating the Store, calls `lz_receive`, and receives `InvalidMint`.

I also tested the proposed atomic transition locally with the real SPL Token program in Surfpool and the compiled `smoke_minter` program. When `set_config` was forced to fail, the whole transaction rolled back: the new multisig was removed, the old mint authority was restored, and the Store remained unchanged.

RECOMMENDATION

1. Add `tasks/smoke/finalizeAuthority.ts` and export a `finalizeMintAuthority` function.

Before sending a transaction, it should verify:

- The recorded OFT and `smoke_minter` programs exist and are executable.
- The mint is owned by the classic SPL Token program and `freezeAuthority` $\equiv$ `null`.
- The recorded OFT Store matches the PDA derived from the recorded OFT program and escrow, is owned by that program, uses `OFTType::Native`, and records the same mint and escrow.
- The canonical `smoke_minter` Store is owned by `smoke_minter` and records the same mint.
- The task wallet is the active `smoke_minter` admin whenever `set_config` is required.
- Each multisig is initialized, has unique non-default members, uses `m` $\equiv$ `1`, and contains exactly the expected members.

Handle the live state as follows:

```
Initial 1-of-3 is live; Store agrees → run the atomic transition
Final 1-of-2 is live; Store is stale → call set_config only
Final 1-of-2 is live; Store agrees → no on-chain write
Anything else → throw before mutation
```

2. In the initial-state branch, create the final multisig, change mint authority, and update the Store in one transaction, in this order:

```
const finalMultisig = Keypair.generate()
const transaction = new Transaction().add(
  SystemProgram.createAccount({
    fromPubkey: taskWallet.publicKey,
    newAccountPubkey: finalMultisig.publicKey,
    lamports: await connection.getMinimumBalanceForRentExemption(MULTISIG_SIZE),
    space: MULTISIG_SIZE,
    programId: TOKEN_PROGRAM_ID,
  }),
  createInitializeMultisigInstruction(
    finalMultisig.publicKey,
    [oftStore, smokeStore],
    1,
  ),
)
```

L-03 · CONTINUED

```

        TOKEN_PROGRAM_ID
    ),
    createSetAuthorityInstruction(
        mint,
        initialMultisig,
        AuthorityType.MintTokens,
        finalMultisig.publicKey,
        [taskWallet],
        TOKEN_PROGRAM_ID
    ),
    buildSetConfigInstruction(finalMultisig.publicKey)
)

await sendAndConfirmTransaction(
    connection,
    transaction,
    [taskWallet, finalMultisig],
    { commitment: 'finalized' }
)

```

Do not add a `FreezeAccount` instruction because this flow requires freeze authority to already be `None`.

3. Immediately after `smoke:minter:set-config`, line 201, import the finalizer and register a new `smoke:mint:finalize-authority` task. Pass the existing instruction encoder into the finalizer:

```

buildSetConfigInstruction: (authority: PublicKey) =>
  minterIx(
    'set_config',
    { params: { smoke_mint: mint, mint_authority_multisig: authority } },
    [
      meta(taskWallet.publicKey, true),
      meta(storePda(), false, true),
      meta(mint),
      meta(authority),
    ]
  )

```

4. After confirmation, re-read the mint, both Stores, and final multisig at `finalized`. Update `mintAuthority` in `deployments/${endpointIdToNetwork(eid)}/OFT.json` only after all values match. Write to a unique temporary file in the same directory and atomically rename it over `OFT.json`.
5. Replace the planned finalization commands:

```

smoke:mint:renounce-freeze
lz:oft:solana:setauthority
smoke:minter:set-config

```

with:

```

smoke:mint:renounce-freeze
smoke:mint:finalize-authority

```

Do not renounce the Store admin until the final state has been verified and `OFT.json` has been updated.

6. Add tests covering the atomic transition, rollback, Store-only repair, stale deployment records, malformed multisigs, invalid Store bindings, and failed post-state verification.



AUDIT COMPLETE

Stay secure.

Thank you for choosing SBSecurity.

WEBSITE

sbsecurity.net

TELEGRAM

[@sbsecurity_audits](https://t.me/sbsecurity_audits)

REPORT

Smoke Ring • September 2026